TransitLink Architecture Blueprint

Project: AI-Driven College Bus Fleet Management System
Document Type: Master Technical Specification & AI Prompt
Team: Rahul (Hardware), Vansh (AI/CV), Siddhant (Backend/DB), Vidhu (Frontend/UI)

> **System Directive:** This document serves as the complete architectural blueprint for "TransitLink." It defines the Database Schema, IoT Edge protocols, AI/CV pipelines, Backend Architecture, Frontend State Management, and specific team member responsibilities.

1. Global System Architecture & Data Flow

1.1 Core Objectives

- **Zero-Touch Tracking:** Eliminate driver dependency using GPS + Geofencing.
- **Sensor Fusion Capacity:** Combine IR break-beam counters (Hardware) + YOLOv8 Seat Classification (AI) for real-time capacity.
- **Digital Queueing:** Afternoon tokens to prevent physical rushing.
- **Intelligent Fallback:** Geospatial PostGIS queries to reroute students to the nearest alternative route.

1.2 Tech Stack Definition

- **Edge IoT (Rahul):** ESP32, C++ (FreeRTOS), IR Break-beam sensors, NEO-6M GPS, SIM800L, MQTT.
- **AI/CV (Vansh):** Python, OpenCV, PyTorch, YOLOv8, DeepSORT, Flask/FastAPI.
- **Backend & DB (Siddhant):** Node.js, Express, Socket.io, Supabase (PostgreSQL 15 + PostGIS), Redis.
- **Frontend Ecosystem (Vidhu):** Next.js 14 (App Router), React Native (Expo), Zustand, React Query.

2. Siddhant: Backend, Database & Cloud Architecture

2.1 Supabase (PostgreSQL) Schema

- `users`: id (UUID), role (enum), default_route_id, home_location (PostGIS POINT).
- `buses`: id, registration_number, max_capacity, status (enum).
- `routes`: id, name, polyline (GeoJSON), is_active.
- `stops`: id, route_id, stop_name, location (PostGIS POINT), geofence_radius_meters.
- `active_trips`: id, bus_id, route_id, start_time, end_time, is_last_bus (Boolean).
- `telemetry` (Partitioned): id, trip_id, location (POINT), speed_kmh, current_pax_count, timestamp.

- `digital_tokens`: id, student_id, trip_id, queue_position, status (enum), created_at.

2.2 PostGIS Geospatial Logic (Fallback Engine)

Function: `find_fallback_route(student_id, current_location)`

- Check if `active_trips` on the student's default route has `is_last_bus = true` AND ended.
- If true, query all stops on ALL other routes.
- Use `ST_Distance` to find the stop nearest to their home.
- Filter routes by `active_trips` where available capacity > 0.

3. Rahul: Hardware, IoT Edge, and Sensors

3.1 C++ FreeRTOS Task Management

- `Task_GPS`: Reads NMEA sentences, updates global state.
- `Task_IR_Counter`: Handles A/B sensor triggers with debounce for directional counting (+1/-1).
- `Task_MQTT_Publish`: Runs every 2 seconds. Connects to broker.

3.2 MQTT JSON Payload Standard

```
{
  "bus_id": "B-402",
  "timestamp": 1698754321,
  "gps": { "lat": 30.3165, "lon": 78.0322, "speed": 42.5, "sats": 8 },
  "pax": { "in_count": 45, "out_count": 12, "net_current": 33 },
  "sos_active": false
}
```

4. Vansh: AI, Computer Vision & Sensor Fusion

4.1 Sensor Fusion Algorithm

- **Problem:** IR sensors miscount overlapping bodies. CV models struggle with occlusion.
- **Logic:** Calculate `expected_empty = max_capacity - IR_net_current`.
- Observe `visual_empty_seats` via YOLOv8 (1 FPS sampling).
- If deviation is high, trigger recalibration: trust CV when stationary, trust IR when moving.

5. Vidhu: Frontend, UI/UX, and Mapping

5.1 Admin Dashboard & Mobile Apps

- **Dispatcher Web:** React-Map-GL for live mapping. Queue manager column view for digital tokens.
- **Student App:** Live tracking with ETA. "JOIN QUEUE" button for afternoon departures. Fallback Alert modals.
- **Driver App:** Zero-touch interface. Giant SOS button and background geofence tracking.

6. Critical Workflows & Edge Cases

- **Last Bus Departure:** Geofence exit triggers backend webhook, checking `is_last_bus`, firing FCM Push Notifications to students.
- **Network Partition:** ESP32 queues telemetry locally. Upon GSM reconnection, bulk publishes data. Frontend interpolates movement smoothly instead of teleporting markers.